7. Image net Classification with Deep CNNs

13 March 2026 17:17

Before this paper → CNN's existed
But they didn't scale to large images

Many reasons → compute → GPUs were not used for DL yet
→ Dataset size → Data sets were small
→ Overfitting i.e. with large models

Paper claims that with large data sets + GPU's + specific training tricks, CNN's can scale

They introduce three design Constraints

Constraint 1 → Data Scale

Model Capacity ∝ Dataset size

An early statement of scaling laws → small data sets → simple models
large datasets → deep models

Constraint 2 → CNN inductive bias

They say CNNs make correct assumptions about images
↳ 1) Locality → nearby pixels are correlated
2) Stationarity → The same feature (edge, corner, texture) can appear anywhere
(so we reuse filters)
That produces weight sharing

Top 1 vs Top 5 error

Top 5 error
Model outputs probabilities for 1000 classes
Take the highest probability classes
if the true class is not in those 5 → error

↓ Model prediction
$\hat{y} = \text{argmax } P(y|x)$
if this prediction ≠ true label → error

→ Final 1000-way softmax

Output layer → 1000 classes → outputs $z_1 \ldots z_{1000}$
↓
then softmax
↓
probability distribution over 1000 classes

→ NON SATURATING NEURONS

Before this paper people used
Sigmoid $\sigma = \frac{1}{1+e^{-x}}$ or tanh } Both saturate
for large positive / negative inputs → gradients ≈ 0
$\frac{d\sigma}{dx} \approx 0$
Training deep networks usually becomes extremely slow

Solution to use ReLU $f(x) = \max(0, x)$

Derivative $f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$ No saturation for positive values
which dramatically improves gradient flow

DROPOUT –
inject noise into the network structure during training

Training step
Randomly remove neurons $h_i = \begin{cases} 0 & \text{with probability } p \\ \frac{h_i}{1-p} & \text{otherwise} \end{cases}$

without dropout
Neurons co-adapt
They rely on other specific neurons
Dropout forces neurons to learn independent features

At each training iteration, every neuron in a given layer (except sometimes the output layer)
has probability rate p (the dropout rate) of being temporarily disabled
these neurons don't contribute to forward pass / backward pass for that specific step

Inference phase Dropout → disabled
to account for the fact that more neurons are active than in training wts/output are scaled to match the expected activation values the network learned
There is stochasticity here

→ prevents co-adaptation → in standard training neurons can develop "complex co-adaptations" Dropout breaks this by making the single presence of any neuron unreliable
→ forces Redundancy → A neuron must learn multiple independent pathways to represent the same information, making it more robust
→ Ensemble effect → can be viewed as training an ensemble of many thinned networks simultaneously

Dropout rates → Starting point is 0.5 for hidden layers & 0.1 to 0.2 for input layers
→ We may need to use larger networks to achieve the same level of learning
→ Takes 2-3 times longer to converge
→ Usually applied after activation functions

Why CNN's have few parameters?

- fully connected layer params = #input × #output
- CNN layer params = $K \times K \times C_{in} \times C_{out}$

} core efficiency gain (Independent of image size)

Image preprocessing

Images have variable resolutions
→ Network requires fixed input

→ Resize image so shortest side = 256
eg original 500×300
resize 426×256

→ Take center crop 256×256
Keep the aspect ratio before cropping

→ Mean Subtraction

$\mu_{pixel} = \frac{1}{N}\sum_{i=1}^{N} x_i$

Then subtract $x' = x - \mu$

} to center data around zero
Better optimization
equivalent to zero mean normalization

The Architecture
↳ contains eight learned layers
↓ five convolutional + 3 fully connected

3.1 ReLU nonlinearity

Sigmoid/tanh → saturating non linearities

ReLU → non saturating non linearity

↳ Networks train really faster by several folds
↓ faster learning has great influence on the performance of large models trained on large datasets

3.2 Training on Multiple GPU's

↳ parallelization scheme puts half neurons on each GPU (+ in total)

↳ additional trick → GPU's communicate in only certain layers

3.3 Local Response Normalization

ReLU's have the desirable property that they do not require input normalization to prevent them from saturating

if atleast some training examples provide a positive input to ReLU training happens in that neuron

they say local normalization scheme aids generalization

$a^i_{x,y}$ → kernel: i
→ neuron activity
→ position (x, y)

then Applying ReLU nonlinearity

↓

The response-normalized activity $b^i_{x,y}$ is given by

$$b^i_{x,y} = a^i_{x,y} \Big/ \left(k + \alpha \sum_{j=\max(0,\, i-n/2)}^{\min(N-1,\, i+n/2)} (a^j_{x,y})^2 \right)^{\beta}$$

$$b^i_{x,y} = a^i_{x,y} \Big/ \left(k + \alpha \sum_{j=\max(0,\, i-n/2)}^{\min(N-1,\, i+n/2)} (a^j_{x,y})^2 \right)^{\beta}$$

n "adjacent" kernel maps at the same spatial position

$N \rightarrow$ total no of kernels in the layer

Ordering of kernel maps is arbitrary + determined before training begins

This response normalization implements a form of lateral inhibition inspired by the type found in real neurons

creating competition for big activities amongst neuron outputs computed using different kernels

k, n, α, β are hyperparameters $\rightarrow$ values are determined using a validation set

Applied after ReLU in some layers

termed this brightness normalization

Image

↓

Conv layer $\rightarrow$ spatial feature extraction

↓

Fully connected $\rightarrow$ semantic discretion

↓

1000 way classifier

So the CNN learns progressively abstract representations

ReLU choice

↳ primarily about optimization speed

eg $\sigma(x) = (1+e^{-x})^{-1}$

$\sigma'(x) = \sigma(x)(1-\sigma(x))$

Max derivative $= 0.25$

for large $|x|$ $\sigma'(x) = 0$

Gradient vanishes

ReLU $f(x) = \max(0, x)$

$f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \le 0 \end{cases}$

for positive region $\frac{\partial f}{\partial x} = 1$

gradients propagate without shrinking

if we remove ReLU & replace with sigmoid

Gradients vanish

training slows dramatically

deeper models fail

$\rightarrow$ What object we are normalizing?

$a^i_{x,y}$ Activation of feature map i at spatial position (x,y) after convolution & ReLU

eg conv layer output

Feature map 1 $\rightarrow$ each feature map is a 2D grid of activations

⋮

Feature map N

eg Feature map 1

(0,0) (0,1) (0,2)

(1,0) (1,1) (1,2)

(2,0) (2,1) (2,2)

each cell corresponds to a specific spatial location in the image

from the equation:

Numerator $a^i_{x,y}$ (original activation)

denominator $k + \alpha \sum (a^j_{x,y})^2$

This computes energy of nearby feature maps

Specifically feature maps around i at the same pixel

example if $n=5$

$i-2$ $i-1$ i $i+1$ $i+2$

Their model was large for one GPU

↓

Split model across two GPUs

didn't duplicate the network

partitioned it!

Simplified structure

GPU1 $\rightarrow$ half feature maps

GPU2 $\rightarrow$ other half

eg if layer has 96 filters

GPU 1 $\rightarrow$ 48 filters

GPU 2 $\rightarrow$ 48 filters

Communication pattern

$\rightarrow$ they deliberately minimized cross GPU communication

Most layers GPU1 $\rightarrow$ GPU1, GPU2 $\rightarrow$ GPU2 } only certain layers exchange activations

early example of model parallelism

Two main strategies today

Data parallelism

Each GPU runs full model on different batches

Model parallelism

Different parts of model on different GPUs

CNNs

Across Channels (lateral inhibition)

edge detector

texture detector

etc

if one activation strongly other get normalized down

Generalization

performance on unseen data

Thus normalization didn't survive long

it got replaced by batch norm, layer norm, group norm

they invented this LRN for solving

$\rightarrow$ large activation magnitudes

$\rightarrow$ feature redundancy

Feature map N eg Feature map 1

(0,0) (0,1) (0,2)
(1,0) (1,1) (1,2)
(2,0) (2,1) (2,2)

each cell corresponds to a specific spatial location in the image

adjacent kernel maps at same spatial position

eg fix a position (x,y)

now look across feature maps

$FM^1 \rightarrow a^1(x,y)$

$FM^5 \rightarrow a^5(x,y)$

Layer normalizes across channels not space

Specifically feature maps around i at the same pixel

example if n=5

$i-2, i-1, i, i+1, i+2$

then square & sum them

Then raise to power β

$(\ )^\beta$ controls normalization strength

This creates competition between feature maps

eg at pixel (x,y)

filter 1 → 10

filter 2 → 9

filter 5 → 0.5

Denominator becomes large

activations are scaled down

But largest one dominates relatively

ENCOURAGES STRONG, SPARSE RESPONSES

they invented this LRN for solving

→ large activation magnitudes

→ feature redundancy

layer Norm

Group Norm

If required extra computation across channels

larger networks → inefficient

scaling problem

Overlapping Pooling

pooling layers in CNNs → summarize outputs of neighbouring groups of neurons in the same feature map

→ generally Representation → keeps strong signals

Information flow → reduces spatial resolution

eg z=2 s=2

→ two params window size = z

Stride = s

A B C D
E F G H
I J K L
M N O P

Pooling windows

[A B] [C D]

[E F] [G H]

Windows don't overlap as z=s

s = z (traditional local pooling)

s < z (Overlapping pooling)

→ slightly lower error

slightly less overfitting

Non overlapping pooling is aggressive down sampling

window size = 3

s = 2

So windows overlap

eg

Window 1 | Window 2

[A B C] [C D E]

[E F G] [G H I]

[I J K] [K L M]

two columns overlap

in overlapping pooling information is more smoothly aggregated

Modern architectures use strided convolutions

3.5 Overall Architecture

8 layers → 5 CNN + 3 fully connected

Output of last fully connected layer is a 1000-way softmax producing distribution over the 1000 class labels

Kernels of 2nd 4th 5th CNN layers are connected only to those kernel maps in previous layer which reside on same GPU

3rd CNN layer connected to all kernel maps in prev layer

Response norm layers follow 1st & 2nd conv layers

Max pooling layers follow both resp norm layers and fifth conv layer

ReLU is applied to every layer

Input image
↓
Conv1
↓
LRN
↓
MaxPool
↓
Conv2
↓
LRN
↓
MaxPool
↓
Conv3
↓
Conv4
↓
Conv5
↓
MaxPool
↓
FC1
↓
FC2
↓
FC3
↓
1000way softmax

Spatial resolution gradually shrinks

Channels grow

Representation becomes

↳ smaller spatially

↳ richer in channels

Why Fully connected layers

↓

they act as global reasoning layers

Why LRN only after 2 layers

early layers detect edges, colors, textures

These features tend to fire simultaneously

LRN prevents them to saturate

Later layers are sparse → LRN not necessary

3 layers without pooling

↓

Allows for large receptive fields without losing resolution

Max pooling layers follow both resp norm layers and fifth conv layer

ReLU is applied to every layer

FC1
↓
FC2
↓
FC3
↓
1000way softmax

they act as global reasoning layers

Allows for large receptive fields without losing resolution

4 Reducing Overfitting

4.1 data augmentation

first form → generating image translations & horizontal reflections

second form → altering the intensities of the RGB channels in training images

$I_{xy} = [I_{xy}^R \; I_{xy}^G \; I_{xy}^B]^T$

we add $[P_1 P_2 P_3][\alpha_1\lambda_1, \alpha_2\lambda_2, \alpha_3\lambda_3]^T$

they perform PCA on set of RGB pixel values

P_i → i^{th} eigenvector

λ_i → eigen values of 3x3 covariance matrix of RGB pixel values

α_i → random variable drawn from gaussian with mean zero and SD 0.1

→ drawn only once for all the pixels of a particular training image until the image is used for training again

Second augmentation

Extract pixel vectors $[I_R I_G I_B]$

Compute covariance Σ_{RGB}

PCA Gives eigen vectors P_i, eigen values λ_i

These represent principal color directions

They perturb each image by adding

$[P_1 P_2 P_3][\alpha_1\lambda_1, \alpha_2\lambda_2, \alpha_3\lambda_3]^T$

$\alpha_i \sim N(0, 0.1)$

Random RGB noise produces unrealistic colors

PCA perturbation natural color correlations

4.2 Dropout

Combining predictions of many diff models is very successful way to reduce test errors

↳ too expensive for big neural networks

Hence here dropout technique is used

5 Details of Learning

SGD used

update rule for weight w

$V_{i+1} = 0.9 V_i - 0.0005 \epsilon w_i - \epsilon \langle \frac{\partial L}{\partial w} |_{w_i} \rangle_{D_i}$

Keeps part of previous update direction

smooths optimization

weight decay term → shrinks weights towards zero

→ standard SGD step

$W_{i+1} = W_i + V_{i+1}$

i → iteration index

V → momentum variable

ϵ → learning rate

$\langle \frac{\partial L}{\partial w} |_{w_i} \rangle_{D_i}$ → average over batch

D_i derivative of the objective w.r.t. w evaluated at w_i

Weight decay also stabilizes optimization here

↳ without it → wts can grow large

loss surface becomes sharp

optimization unstable

Large initial wts cause → very large activations

unstable gradients

ReLU
+good initialization
+momentum
+weight decay

} Gradient flow lives

why learning rate decay?

early training → large steps fast progress

Later training → Smaller steps fine tuning

initialized neuron biases in second, 4th, 5th as well as fully connected layers by providing ReLUs with positive inputs

in other layers Bias = 0

Model learned Meaningful Visual similarity

The network learnt a semantic embedding space

Images close in feature space often share semantic meaning

1 Depth matters

→ they performed Ablation → removing any conv layer increases error

2 CNNS scaling works

→ Large CNN's trained on large datasets work well

3 Supervised training alone was enough

4 Computational bottleneck

↳ bigger models → bigger compute → likely better results

Five ideas compressed

↳ Deep CNNs can scale to ImageNet

↳ ReLU solves optimization issues

↳ Regularization is essential (augmentation, dropout, weight decay)

↳ Architecture pattern → conv → conv → conv → pool

↳ Deep CNNs can scale to imagenet

↳ ReLU solves optimization issues

↳ Regularization is essential (augmentation, dropout, weight decay)

↳ Architecture pattern → conv → conv → conv → pool → large receptive fields

↓

Fully connected

↓

softmax

↳ Hardware and Algo together matter

- GPU compute
- dataset
- optimization tricks
- architecture design

Historical impact

↳ within 3 yrs

- VGGNet
- GoogLeNet
- ResNet